FORMATO PARA ELABORACIÓN DE TALLERES ESTUDIANTILES	Departamento de Ciencias de la Computación
	Carrera: Ingeniería en Tecnologías de la Información Asignatura: Metodología de Desarrollo de Software NRC: 30693
	Apellidos y nombres del estudiante: JARAMILLO APUNTE CHRISTOPHER MATEO

ANTECEDENTES

OBJETIVO:

Desarrollar un análisis y fundamentar las metodologías de desarrollo de software, con la definición correspondiente de cada una de ellas, con el fin de comprender sus respectivas aplicaciones en el ciclo de vida de software comprendiendo sus aportes en la eficacia y calidad del proceso fundamentando las justificaciones en libros o guías de estudio.

DESARROLLO:

1. **Seleccione una o más de una de las siguientes afirmaciones: Identifique los tipos de clases de metodologías de desarrollo de software.**
   a) Estructurada
   b) Orientada a Objetos

Justificación:

Pressman explica que los modelos estructurados, como el modelo en cascada, se basan en fases secuenciales bien definidas (análisis, diseño, codificación, pruebas y mantenimiento). "Los modelos estructurados de proceso de software siguen un enfoque secuencial, donde cada fase debe completarse antes de iniciar la siguiente, proporcionando control y disciplina en proyectos grandes." (Pressman, 2010, Capítulo 3).

Pressman también señala que el enfoque orientado a objetos es una metodología que organiza el software en clases y objetos, promoviendo la modularidad, reutilización y mantenibilidad. "La orientación a objetos proporciona un marco metodológico que facilita el desarrollo de sistemas complejos, al organizar el software en clases y objetos que encapsulan datos y comportamiento." (Pressman, 2010, Capítulo 3).

2. **Identifique la definición de metodología de desarrollo de software**
   a) Un conjunto de técnicas y procedimientos organizados en fases para el desarrollo de productos software, de manera eficaz, y abarca el ciclo de vida del mismo.

Justificación:

Se define una metodología de desarrollo de software como un marco de trabajo integral que guía todas las actividades del ciclo de vida del software. No se limita a herramientas o lenguajes, sino que incluye tareas, hitos y productos organizados en fases, bajo una filosofía de gestión que asegura eficacia y calidad.

3. **Complemente la siguiente afirmación: Las metodologías orientadas a objetos centran su análisis, diseño e implementación en…**

Los objetos.

Justificación:

Las metodologías orientadas a objetos, se fundamentan en el paradigma de que el software debe modelarse como un conjunto de objetos que interactúan entre sí, en lugar de una secuencia de funciones o procesos. Esta elección es la correcta porque el objeto es la unidad fundamental que encapsula tanto la estructura (datos/atributos) como el comportamiento.

Mientras que las metodologías estructuradas separan los procesos de los datos, la orientación a objetos los unifica en estas entidades, permitiendo que el análisis y el diseño se centren en las clases y objetos detectados en el dominio del problema (Pressman, 2010).

4. **Identifique los principios del "manifiesto ágil"**

- Es más importante crear un producto software que funcione que escribir documentación exhaustiva.
- La colaboración con el cliente debe prevalecer sobre la negociación de contratos.

Justificación:

El Software funcionando sobre documentación extensiva no implica que la documentación no sea necesaria, sino que el objetivo principal es entregar un producto operativo que aporte valor real al usuario, evitando que el exceso de burocracia técnica retrase el desarrollo.

La colaboración con el cliente sobre negociación contractual y las metodologías ágiles proponen una relación de confianza y trabajo conjunto con el cliente durante todo el ciclo de vida, en lugar de limitarse a cumplir estrictamente un contrato estático que no permite adaptarse a cambios en el mercado o en los requisitos(Pressman, 2010).

5. **Empareje las definiciones de los diagramas de UML con los términos correspondientes**

1. Utilizados para denotar los nombres de los componentes (clase), sus responsabilidades y los nombres de sus componentes colaboradores	A. Gráficos de estructura
2. Diseñados y utilizados para representar modelos conceptuales de datos almacenados en repositorios de información	B. Diagramas de Implementación
3. Se utilizan para representar un conjunto de nodos (físicos) y sus interrelaciones y, por lo tanto, para modelar los aspectos físicos del software	C. Diagramas entidad relación
4. Se utilizan para describir la estructura de llamada de los programas.	D. Tarjetas clase-responsabilidad (CRC)

Respuestas: 1D,2C,3B,4A

Justificación:

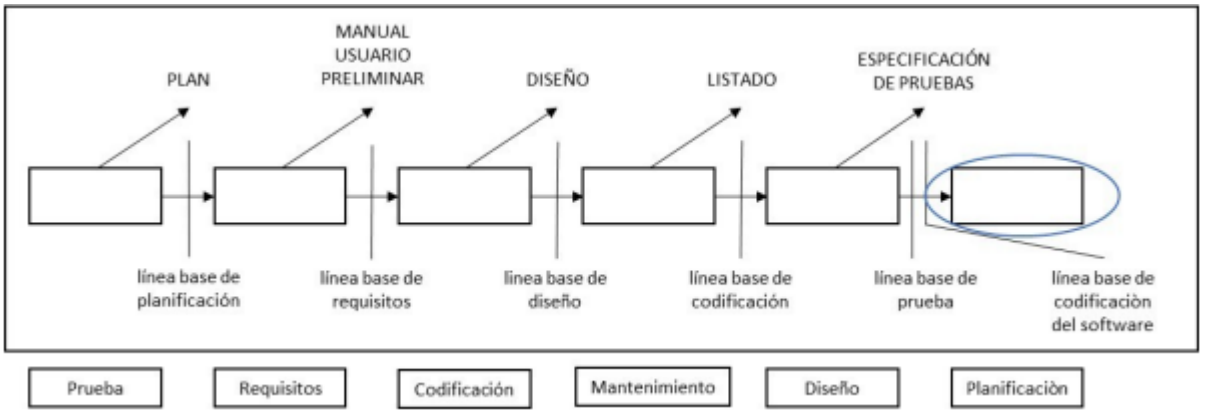
1 - D (Tarjetas CRC): Las tarjetas Clase-Responsabilidad-Colaborador son una técnica de diseño donde se identifica qué hace cada clase (responsabilidad) y con quién necesita interactuar (colaboradores).
2 - C (Diagramas Entidad-Relación): Se usan para modelos conceptuales de datos almacenados en repositorios. Los diagramas ER representan entidades, atributos y relaciones en bases de datos.
3 - B (Diagramas de Implementación/Despliegue): En la sección de diagramas físicos de UML, se especifica que estos representan la arquitectura física del sistema, es decir, los nodos (servidores, PCs, dispositivos) y cómo se conectan entre sí para ejecutar el software.
4 - A (Gráficos de Estructura): Estos pertenecen al análisis y diseño estructurado. Se definen como la representación de la jerarquía y el flujo de control (llamadas) entre módulos de un programa.

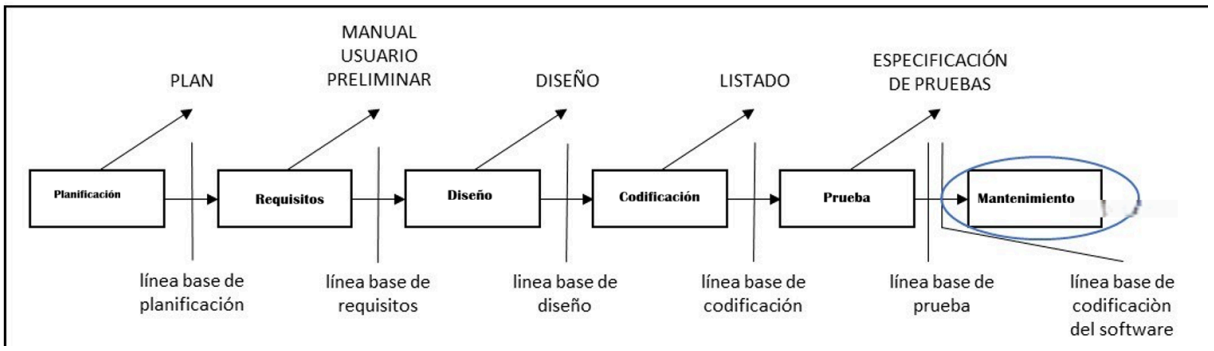
6. **Las fases del proceso de ingeniería de requisitos son**

Educción, análisis / negociación, especificación y validación

Fase	Justificación según Pressman/Sommerville
Elicitación (Educación)	El proceso de descubrir los requisitos mediante entrevistas y talleres.
Análisis	Refinar los requisitos para entender las interdependencias y conflictos.
Especificación	Creación del documento formal (ERS) que describe el sistema.
Validación	Revisión técnica para asegurar que los requisitos no tengan errores.

7. **Complete el siguiente gráfico que representa la línea base y los elementos de la configuración del software.**





1. Planificación (Primer recuadro)

- Es el punto de partida de todo proyecto de software.

2. Requisitos (Segundo recuadro)

- Debido a que en los requisitos se define qué hará el sistema y cómo interactuará el usuario con él.

3. Diseño (Tercer recuadro)

- Esta fase traduce lo que el cliente quiere (requisitos) a instrucciones técnicas.

4. Codificación (Cuarto recuadro)

- Hace referencia al listado de código fuente o *scripts* escritos por los programadores. Es la etapa donde se construye o programa el software.

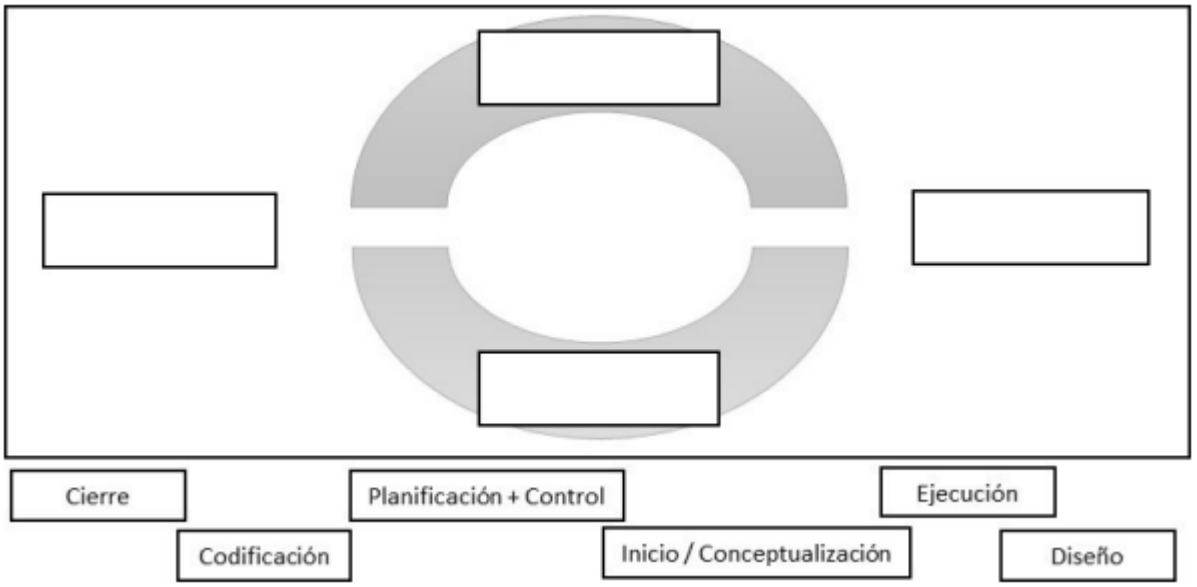
5. Prueba (Quinto recuadro)

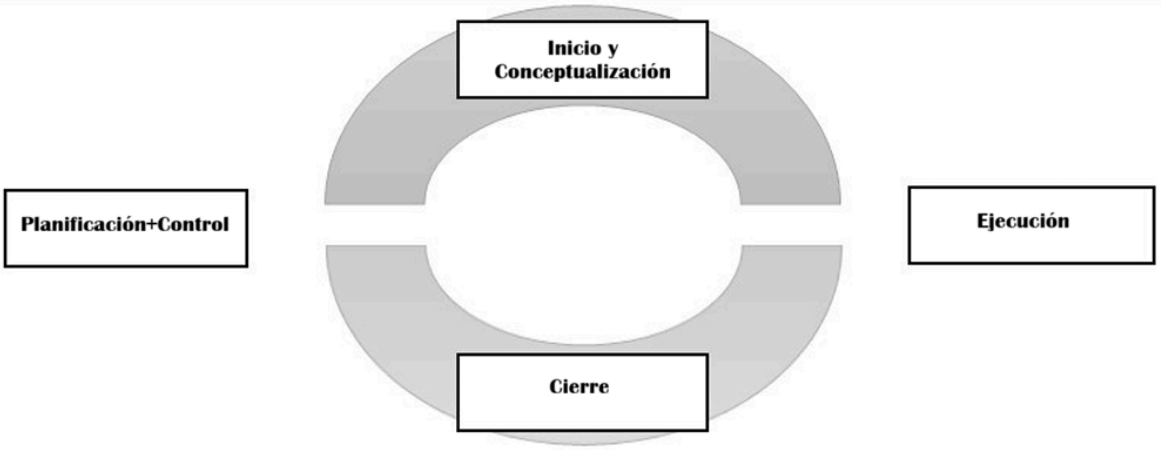
- Aquí se valida que el código creado en la fase anterior funcione y cumpla con los requisitos.

6. Mantenimiento (Sexto recuadro)

- Una vez que el software es probado y entregado, entra en su fase operativa o de Mantenimiento. Cualquier cambio futuro (corrección de errores o nuevas funciones) se controlará a partir de la línea base del producto de software final entregado

8. **El siguiente gráfico representa las fases de una metodología de gestión de proyecto de desarrollo de software, revisada en clases. Complete el gráfico con el nombre de las fases.**





1. Inicio / Conceptualización

Es el punto de partida del proyecto. Aquí se define a grandes rasgos qué se va a hacer, se aprueba el proyecto (acta de constitución) y se identifican a los interesados.

2. Planificación + Control y Ejecución

El gráfico muestra estas dos fases dentro de un ciclo cerrado porque son iterativas y ocurren casi en paralelo. Generalmente, la Planificación + Control guía el trabajo, y la Ejecución, que es donde se realiza el trabajo real. Mientras el equipo "ejecuta", el director del proyecto "controla" y ajusta la "planificación", creando este ciclo continuo hasta terminar el trabajo.

3.Cierre

Es la culminación del proyecto. Aquí se entregan los resultados finales al cliente, se liberan los recursos y se documentan las lecciones aprendidas.

CONCLUSIONES:

- Importancia de la Gestión de Configuración: Se determinó que la Gestión de Configuración de Software (SCM) es un proceso transversal crítico que garantiza la integridad y trazabilidad del producto. Sin una disciplina que identifique y controle los cambios, el desarrollo de software a gran escala se vuelve propenso a errores de versiones y pérdida de información técnica valiosa.
- Centralidad del Paradigma Orientado a Objetos: El análisis del enfoque orientado a objetos confirma que esta metodología facilita la representación de problemas complejos al centrar el diseño en objetos. Al unificar datos y comportamiento, se logra un sistema más modular, reutilizable y fácil de mantener en comparación con las metodologías estructuradas tradicionales.
- El análisis realizado demuestra que las metodologías de desarrollo de software, tanto estructuradas como orientadas a objetos, ofrecen enfoques distintos pero complementarios: las primeras aportan disciplina y control mediante fases secuenciales, mientras que las segundas integran datos y procesos en objetos, favoreciendo la modularidad y reutilización.
- La definición de metodología de desarrollo de software, según Pressman, confirma que estas constituyen un marco integral de técnicas y procedimientos organizados en fases, lo que asegura eficacia y calidad en el ciclo de vida del software.

RECOMENDACIONES:

- Implementación de Herramientas de Control de Versiones: Se recomienda integrar herramientas de SCM (como Git) desde las etapas tempranas del ciclo de vida para automatizar el control de cambios y facilitar la auditoría de los elementos de configuración, minimizando el riesgo de conflictos en entornos de trabajo colaborativo.
- Reforzamiento en Modelado UML: Para maximizar los beneficios de las metodologías orientadas a objetos, es aconsejable profundizar en el uso de Lenguaje de Modelado Unificado (UML). Esto permitirá documentar de forma estandarizada la interacción entre objetos antes de proceder a la implementación en código.
- Adopción Gradual de Marcos de Trabajo Ágiles: Para proyectos que enfrentan una alta volatilidad en sus requerimientos, se sugiere la adopción de frameworks como Scrum o Kanban. Estos marcos permiten aplicar de manera práctica los principios del manifiesto ágil, fomentando entregas incrementales y una comunicación constante con los interesados.

INTEGRANTES	
N°	**Apellidos y Nombres**
1	Cristopher Mateo Jaramillo Apunte
2	Soza Morales Melany Valeria
3	Velasquez Alay Mery Stephanie
4	Verdezoto Armijos Miguel Alejandro

BIBLIOGRAFÍAS:

Pressman, R. 7. (2005). (M. Hill, Editor) Obtenido de https://www.mlsu.ac.in/econtents/16_EBOOK 7th_ed_software_engineering_a_practitioners_approach_by_roger_s._pressman_.pdf

Pressman, R. S. (2010). *Ingeniería del Software: Un enfoque práctico* (7.ª ed.). McGraw-Hill Interamericana.